

Construção de um Analisador Sintático SLR para uma Linguagem Hipotética

Autor: Matheus Slama

Instituição: Universidade Federal da Fronteira Sul - UFFS

Disciplina: Construção de Compiladores

Projeto: Reconhecedor Sintático

Resumo

Este trabalho apresenta o desenvolvimento de um analisador sintático para uma linguagem hipotética simples, construída a partir dos tokens reconhecidos por um analisador léxico desenvolvido anteriormente. O objetivo principal é implementar um reconhecedor sintático baseado no método SLR, capaz de verificar se uma sequência de tokens pertence ou não à linguagem definida por uma gramática livre de contexto. Para isso, foram realizadas etapas como a definição e fatoração da gramática, cálculo dos conjuntos FIRST e FOLLOW, construção dos itens LR(0), geração da tabela SLR e execução do algoritmo de análise sintática shift-reduce. Além do reconhecimento sintático, o projeto também inclui uma análise semântica simples, geração de código intermediário e uma etapa básica de otimização. Como validação, foi utilizado um estudo de caso com uma entrada textual processada pelo analisador léxico, demonstrando a integração entre as etapas do pequeno pipeline de compilação desenvolvido.

Introdução

Reconhecedores sintáticos são partes fundamentais no processo de construção de compiladores. Depois que o analisador léxico identifica os lexemas e os transforma em tokens, o analisador sintático recebe essa sequência e verifica se ela respeita as regras estruturais da linguagem. Em outras palavras, a análise sintática é responsável por conferir se os tokens aparecem em uma ordem válida, de acordo com a gramática definida.

Essa etapa é importante porque uma linguagem de programação não é formada apenas por palavras válidas, mas também por estruturas corretas. Uma palavra reservada pode existir na linguagem, assim como identificadores e operadores, mas isso não significa que qualquer combinação desses elementos forme um programa válido. Cabe ao reconhecedor sintático identificar se a combinação dos tokens produzidos pelo léxico forma comandos aceitos pela linguagem.

Na prática, analisadores sintáticos são utilizados em compiladores, interpretadores, ferramentas de análise de código, validadores de linguagem, ambientes de desenvolvimento e também em aplicações educacionais voltadas ao ensino de linguagens formais e compiladores. Eles podem produzir apenas uma resposta de aceite ou erro, mas também podem gerar estruturas intermediárias, como árvores sintáticas, árvores abstratas, registros de reduções ou informações que serão utilizadas nas etapas semânticas e de geração de código.

O problema abordado neste trabalho consiste em construir um reconhecedor sintático para uma linguagem hipotética composta por palavras reservadas e identificadores. A linguagem foi definida a partir dos tokens reconhecidos no analisador léxico do projeto anterior, utilizando palavras como *se*, *sai* e *foi*, além do token *ID*, que representa identificadores.

O objetivo do trabalho é desenvolver um analisador sintático baseado em tabela SLR, capaz de reconhecer comandos válidos da linguagem, registrar os passos do processo de análise e integrar essa etapa com outras fases simplificadas de um compilador. Dessa forma, busca-se demonstrar, de maneira prática, como os conceitos estudados em sala podem ser aplicados na construção de um reconhecedor sintático funcional.

Referencial Teórico

Uma gramática livre de contexto, também chamada de GLC, é uma forma de descrever a estrutura sintática de uma linguagem. Ela é composta por um conjunto de terminais, não-terminais, produções e um símbolo inicial. Os terminais representam os símbolos básicos da linguagem, como tokens vindos do

analisador léxico. Os não-terminais representam estruturas mais abstratas, como comandos, expressões ou listas de comandos. As produções indicam como essas estruturas podem ser formadas.

No contexto deste trabalho, a gramática define como os comandos da linguagem podem ser construídos a partir dos tokens disponíveis. Como a linguagem é simples, os comandos envolvem basicamente identificadores e palavras reservadas. Ainda assim, mesmo uma linguagem reduzida exige uma gramática bem organizada para que o processo de reconhecimento sintático funcione corretamente.

Antes da construção da tabela sintática, podem ser necessárias transformações na gramática. Uma dessas transformações é a eliminação de símbolos inúteis, que remove símbolos que não contribuem para a geração de sentenças válidas ou que não podem ser alcançados a partir do símbolo inicial. Outra transformação importante é a fatoração, utilizada quando produções diferentes possuem prefixos comuns. A fatoração reorganiza essas produções para deixar a gramática mais adequada ao processo de análise.

Também são utilizados os conjuntos FIRST e FOLLOW. O conjunto FIRST de um símbolo indica quais terminais podem aparecer no início de cadeias derivadas a partir desse símbolo. Já o conjunto FOLLOW de um não-terminal indica quais terminais podem aparecer imediatamente após ele em alguma derivação válida. Esses conjuntos são muito utilizados em técnicas de análise sintática, pois ajudam a tomar decisões durante o processo de parsing.

O método SLR, abreviação de Simple LR, é uma técnica de análise sintática ascendente. Diferentemente de analisadores descendentes, que tentam construir a derivação a partir do símbolo inicial, o analisador ascendente parte dos tokens da entrada e tenta reduzi-los até chegar ao símbolo inicial da gramática. Para isso, o método SLR utiliza itens LR(0), que representam estados do reconhecimento, e os conjuntos FOLLOW, que auxiliam na escolha das reduções.

A tabela SLR é composta por duas partes principais: ACTION e GOTO. A parte ACTION é utilizada quando o próximo símbolo da entrada é um terminal. Ela indica se o parser deve realizar um deslocamento, uma redução, aceitar a entrada ou indicar erro. A parte GOTO é utilizada após reduções, indicando para qual estado o analisador deve ir ao reconhecer determinado não-terminal.

O algoritmo utilizado pelo parser é conhecido como shift-reduce. A ação shift desloca um token da entrada para a pilha, junto com o estado correspondente. A ação reduce identifica que o topo da pilha corresponde ao lado direito de uma produção e substitui essa sequência pelo não-terminal do lado esquerdo. Quando a tabela indica a ação acc, a sentença é aceita. Caso não exista ação válida para o estado atual e o símbolo de entrada, ocorre um erro sintático.

Implementação e Resultados

O projeto foi organizado em etapas, simulando um pequeno fluxo de compilação. A entrada inicial é fornecida em um arquivo de texto, com um token ou lexema por linha. Essa entrada é processada pelo analisador léxico, que gera uma fita de tokens. Em seguida, essa fita é utilizada como entrada para o analisador sintático.

A linguagem reconhecida utiliza as palavras reservadas se, sai e foi, além de identificadores representados pelo token ID. A gramática original definida para o trabalho foi:

```
P ::= LISTA
LISTA ::= COMANDO LISTA | COMANDO
COMANDO ::= se ID foi | se ID sai | sai ID | foi ID | ID
```

Essa gramática permite representar uma lista de comandos. Um comando pode ser formado por uma estrutura iniciada pela palavra se, seguida de um identificador e uma ação, ou por comandos mais simples iniciados por sai, foi ou apenas por um identificador.

Após a análise inicial, a gramática foi fatorada para organizar melhor as produções com prefixos comuns. A versão fatorada utilizada pelo parser ficou da seguinte forma:

```
P ::= LISTA
LISTA ::= COMANDO LISTA_TAIL
LISTA_TAIL ::= COMANDO LISTA_TAIL | ε
COMANDO ::= se ID ACAO | sai ID | foi ID | ID
ACAO ::= foi | sai
```

A fatoração separou o prefixo comum das produções que começavam com se ID, criando o não-terminal ACAO. Além disso, foi criado o não-terminal LISTA_TAIL, responsável por representar a continuação de uma sequência de comandos. Isso evita a repetição direta da alternativa LISTA ::= COMANDO LISTA | COMANDO e torna a gramática mais organizada para a construção do reconhecedor.

Na implementação do analisador sintático, primeiro o programa carrega a gramática e identifica os símbolos terminais e não-terminais. Em seguida, verifica a existência de símbolos inúteis, calcula os conjuntos FIRST e FOLLOW e constrói a coleção canônica de itens LR(0). Com esses itens, são geradas as transições entre estados e, por fim, a tabela SLR.

Depois da construção da tabela, o parser executa o algoritmo shift-reduce sobre a fita de tokens produzida pelo analisador léxico. Durante esse processo, são registrados os passos da análise, incluindo o conteúdo da pilha, a entrada restante e a ação executada em cada etapa. Esses registros são importantes porque permitem acompanhar o funcionamento do reconhecedor e verificar onde um possível erro sintático ocorre.

Como estudo de caso, foi utilizada a seguinte entrada:

```
nada
se
tudo
foi
vai
```

O analisador léxico transforma essa entrada na seguinte fita de tokens:

```
ID se ID foi ID $
```

O símbolo \$ representa o fim da entrada. A partir dessa fita, o analisador sintático consulta a tabela SLR e executa as ações de deslocamento e redução necessárias. O resultado obtido foi aceite sintático, pois a entrada corresponde a uma lista de comandos válidos da linguagem.

A entrada pode ser interpretada como três comandos. O primeiro comando é formado apenas por um identificador, representado por nada. O segundo comando segue a produção se ID ACAO, sendo composto por se tudo foi. O terceiro comando também é formado apenas por um identificador, representado por vai. Dessa forma, a sequência completa respeita a gramática definida.

Os principais arquivos gerados pela etapa sintática ficam armazenados em uma pasta de saídas. Entre eles estão a fita produzida pelo analisador léxico, a tabela de símbolos, os conjuntos FIRST e FOLLOW, os itens válidos, as transições, a tabela SLR, os passos do parser e o resultado final da análise sintática. Essa organização facilita a validação do projeto, pois permite observar separadamente cada parte do processo.

Além do reconhecimento sintático, também foi implementada uma análise semântica simples. A regra definida para essa etapa foi que os identificadores da entrada devem ser únicos. Para isso, a tabela de símbolos é enriquecida com informações adicionais, como categoria semântica, papel semântico, comando associado, símbolo semântico e validade semântica. Caso um mesmo identificador apareça mais de uma vez, a entrada pode até ser aceita sintaticamente, mas será rejeitada semanticamente.

No estudo de caso utilizado, os identificadores nada, tudo e vai são diferentes. Portanto, a análise semântica também resulta em aceite. Um exemplo de entrada que poderia gerar erro semântico seria:

```
nada  
sai  
nada
```

Nesse caso, a sequência pode estar correta do ponto de vista sintático, mas o identificador nada aparece mais de uma vez, violando a regra semântica definida para o projeto.

Na etapa seguinte, o projeto gera um código intermediário simples para os comandos reconhecidos. Para comandos formados apenas por identificadores, são geradas instruções de declaração e uma atribuição redundante, utilizada posteriormente para demonstrar a etapa de otimização. Para comandos condicionais, é gerada uma instrução intermediária do tipo IF.

Um exemplo de código intermediário produzido para o estudo de caso é:

```
DECL id_nada  
id_nada = id_nada  
IF id_tudo THEN foi  
NOP  
DECL id_vai  
id_vai = id_vai
```

Por fim, é aplicada uma otimização simples sobre esse código intermediário. A estratégia implementada remove instruções NOP e atribuições redundantes do tipo $x = x$. Após a otimização, o código intermediário mantém apenas as instruções consideradas úteis:

```
DECL id_nada  
IF id_tudo THEN foi  
DECL id_vai
```

Com isso, o projeto demonstra não apenas o reconhecimento sintático, mas também uma continuação simplificada do fluxo de compilação, passando por análise semântica, geração de código intermediário e otimização.

O projeto pode ser testado alterando o arquivo de entrada e executando o programa principal com o comando:

```
node main.js
```

Entradas sintaticamente inválidas, como se foi nada, produzem erro sintático, pois não seguem nenhuma produção válida da gramática. Já entradas com identificadores repetidos podem ser aceitas pelo analisador sintático, mas rejeitadas na etapa semântica. Essa separação mostra a diferença entre erro sintático e erro semântico, reforçando a importância de cada fase do processo de compilação.

Conclusões

Este trabalho apresentou a construção de um reconhecedor sintático SLR para uma linguagem hipotética simples. Foram desenvolvidas as etapas necessárias para a definição da gramática, fatoração das produções, cálculo dos conjuntos FIRST e FOLLOW, construção dos itens LR(0), geração da tabela SLR e execução do algoritmo shift-reduce.

A implementação permitiu validar o funcionamento do analisador sintático a partir de uma entrada processada pelo analisador léxico. O estudo de caso demonstrou que a fita de tokens gerada foi corretamente reconhecida pela tabela SLR, resultando em aceite sintático. Além disso, o projeto foi expandido com uma análise semântica simples, geração de código intermediário e uma etapa básica de otimização.

Entre as principais dificuldades encontradas estão a organização correta da gramática, a montagem da tabela SLR e a integração entre as saídas de uma etapa e as entradas da etapa seguinte. A construção dos itens LR(0) e das ações de redução também exige atenção, pois pequenos erros podem gerar conflitos ou impedir o reconhecimento correto das sentenças.

Como resultado final, foi obtido um pequeno pipeline de compilação, capaz de partir de uma entrada textual, gerar tokens, realizar o reconhecimento sintático, validar uma regra semântica, produzir código intermediário e aplicar uma otimização simples. Mesmo sendo uma linguagem reduzida, o projeto permite visualizar de forma prática como diferentes conceitos de compiladores se conectam.

Como continuidade, o trabalho poderia ser expandido com uma linguagem maior, incluindo expressões aritméticas, comandos de repetição, atribuições e blocos de comandos. Também seria interessante gerar uma árvore sintática ou uma árvore sintática abstrata, tornando mais clara a relação entre a entrada reconhecida e sua estrutura interna. Outra melhoria possível seria a criação de mensagens de erro mais detalhadas, indicando o token inesperado, o estado do parser e as produções esperadas.

Por fim, a implementação também poderia ser adaptada para fins didáticos no ensino de compiladores. Por exemplo, o sistema poderia exibir visualmente a pilha, a entrada restante e a ação executada em cada passo do parser. Isso ajudaria estudantes a compreender melhor o funcionamento de analisadores ascendentes e a relação entre gramáticas, tabelas sintáticas e reconhecimento de sentenças.